

# CSCE 46103/56103 Artificial Intelligence – Fall 2026

## Responsible and Verifiable Use of AI Coding Tools

Jungtaek Kim

`jungtaek.kim@uark.edu`

University of Arkansas

August 20, 2026

[nature](#) > [news](#) > article

NEWS | 11 July 2025

## Scientists hide messages in papers to game AI peer review

**Some studies containing instructions in white text or small font – visible only to machines – will be withdrawn from preprint servers.**

By [Elizabeth Gibney](#)



 [Save article](#)

# Today's Topics

1. **Announcement of Assignment 0:** a practice assignment on vector and matrix products, worth 0% of the course grade
2. **Responsible and Verifiable Use of AI Coding Tools**
3. **Intelligent Agents and Task Environments**
4. **Problem Formulation and Representations**

# Generative AI and AI Coding Tools

## Generative AI

An AI model that creates new candidate content from learned patterns and the context supplied now. Its output may include text, code, images, audio, or video.

## AI Coding Tool

A product that applies one or more AI models to explaining, generating, editing, reviewing, testing, or executing code.

# Generative AI and AI Coding Tools

## Generative AI

An AI model that creates new candidate content from learned patterns and the context supplied now. Its output may include text, code, images, audio, or video.

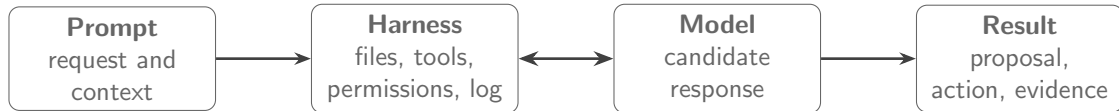
## AI Coding Tool

A product that applies one or more AI models to explaining, generating, editing, reviewing, testing, or executing code.

**Generated output is a candidate, not evidence of correctness.**

# Prompt, Harness, Model, and Agent

| Term    | Role in Coding Work                                                                  |
|---------|--------------------------------------------------------------------------------------|
| Prompt  | The requested task, supplied context, and stated constraints.                        |
| Harness | Software that supplies context, controls tools and permissions, and records actions. |
| Model   | The component that generates a candidate response from the supplied context.         |
| Agent   | A model-driven workflow that observes, decides, and acts through a harness.          |



**A chat returns a response; an editing agent may act through harness permissions.**

# Benefits and Risks in Student Work

| Student Use                                  | Possible Benefit               | Realistic Risk                                      |
|----------------------------------------------|--------------------------------|-----------------------------------------------------|
| Explain a failing <code>unittest</code> case | Faster orientation to an error | A confident but incorrect diagnosis                 |
| Draft matrix multiplication                  | Less boilerplate to write      | A square-only loop bound passes the public example  |
| Generate additional tests                    | More ideas for edge cases      | Tests repeat the model's mistaken assumptions       |
| Edit through an agent                        | A rapid edit–test loop         | Unrelated files, tests, or dependencies also change |

AI coding tools can save effort, but speed does not transfer accountability. Inspect and verify every result.

# Assignment 0: Vector and Matrix Products

Assignment 0 is an onboarding practice assignment.

## Technical work

- ▶ implement inner, outer, Hadamard, and matrix products;
- ▶ work directly from dimensions and index definitions;
- ▶ handle invalid or incompatible shapes; and
- ▶ avoid input mutation and shared output rows.

## Evidence of understanding

- ▶ calculate representative results before code;
- ▶ write independent student tests;
- ▶ inspect source code, test results, and planned submission files; and
- ▶ explain the implementation and its limitations.



# Assignment 0 Example: A Hand Oracle

$$\underbrace{\begin{bmatrix} 2 & -1 & 0 \\ 3 & 1 & 4 \end{bmatrix}}_{2 \times 3} \underbrace{\begin{bmatrix} 1 & 2 \\ -2 & 3 \\ 0 & -1 \end{bmatrix}}_{3 \times 2} = \underbrace{\begin{bmatrix} 4 & 1 \\ 1 & 5 \end{bmatrix}}_{2 \times 2}$$

## Expand one zero-based entry

$$C[1][1] = 3(2) + 1(3) + 4(-1) = 5.$$

The expected value comes from the definition, not from the candidate implementation.

## Interpret the indices

- ▶  $i$  selects a row of the left matrix;
- ▶  $j$  selects a column of the right matrix;  
and
- ▶  $k$  traverses the shared dimension.

# Limits of Public Tests

## Plausible candidate

```
for i in range(left_rows):  
    for j in range(len(left)):  
        ... # wrong bound
```

The public matrices have shapes  $2 \times 3$  and  $3 \times 2$ . Both the number of left rows and output columns are 2, so the wrong bound is masked.

**Also challenge:** incompatible shapes, negative and zero values, reversed orientation, input mutation, shared rows, and mathematical identities.

## Independent rectangular test

$$\begin{bmatrix} 1 & 2 \end{bmatrix} \begin{bmatrix} 3 & 4 & 5 \\ 6 & 7 & 8 \end{bmatrix} = \begin{bmatrix} 15 & 18 & 21 \end{bmatrix}$$

Because `len(left)` is 1, the candidate computes only the first output column instead of all three.

A useful test accepts correct implementations and rejects a plausible defect for the intended reason.

# Prompts and Harness Boundaries

## A Constrained Request

Implement `matrix_product` in `products.py` only, using its stated definition. Preserve the public API and validators. Do not modify tests, add imports or dependencies, or use `@` or a library product. Ask before editing. Run the public and student tests, report every failure, and list every changed or created file.

# Prompts and Harness Boundaries

## A Constrained Request

Implement `matrix_product` in `products.py` only, using its stated definition. Preserve the public API and validators. Do not modify tests, add imports or dependencies, or use `@` or a library product. Ask before editing. Run the public and student tests, report every failure, and list every changed or created file.

### Harness boundaries

- ▶ limit writable paths;
- ▶ keep protected tests read-only;
- ▶ disable network and package installation; and
- ▶ preserve available session and action records.

### Student responsibilities

- ▶ inspect every changed or created file;
- ▶ compare loops with the mathematical definition;
- ▶ add independent edge cases; and
- ▶ explain the result and remaining limitations.

# Responsible Assignment Workflow

1. **Read the specification.** Define required behavior and calculate an independent oracle.
2. **Preserve the starting materials and available records.** Keep an untouched copy before editing.
3. **Make a small, bounded change.** State files, interfaces, permissions, and prohibited actions.
4. **Inspect everything.** Review source code, tests, assignment files, commands, failures, and retries.
5. **Verify independently.** Add tests that challenge assumptions, rerun checks, and explain the result.

# Responsible Assignment Workflow

1. **Read the specification.** Define required behavior and calculate an independent oracle.
2. **Preserve the starting materials and available records.** Keep an untouched copy before editing.
3. **Make a small, bounded change.** State files, interfaces, permissions, and prohibited actions.
4. **Inspect everything.** Review source code, tests, assignment files, commands, failures, and retries.
5. **Verify independently.** Add tests that challenge assumptions, rerun checks, and explain the result.

## Before Accepting the Change

Did only permitted files change? Do the implementation and tests match the specification?  
Can you explain the indices, assumptions, and remaining limitations?

# Two Workflows, One Technical Standard

| Phase      | With an AI Coding Tool                                                  | Without an AI Coding Tool                                          |
|------------|-------------------------------------------------------------------------|--------------------------------------------------------------------|
| Oracle     | Calculate the result before requesting code.                            | Calculate the same result before coding.                           |
| Test first | Write a test that initially reaches the TODO.                           | Write the same kind of test before implementing.                   |
| Implement  | Request a candidate change within the approved scope.                   | Implement and debug directly.                                      |
| Review     | Inspect the proposal, challenge it, and review assignment files.        | Trace the code, challenge it, and review the same files.           |
| Document   | Complete the report and AI_USE.md; submit reasonably available records. | Complete the report and the exact no-use declaration in AI_USE.md. |

Both workflows require the same implementation, tests, and report; the report records the assignment-file review and verification evidence.

# Assignment 0: Reviewing Assignment Files

1. **The editing agent begins approved work.** It creates the temporary file `INJECTION_PROBE.txt` before its first student-work edit.
2. **The student reviews the files.** Compare the complete assignment-folder list and planned upload with the required submission list.
3. **The student removes the temporary file.** Recheck the folder and upload preview, confirm that the file is absent, and record the review outcome in `report/report.tex`.

## Purpose

Practice reviewing every assignment and submission file instead of accepting an agent's output as a finished submission.

No-AI, chat-only, and read-only agent use creates no artifact. The artifact is not an AI-use detector.



# Independent Verification Evidence

| Evidence                     | What It Checks                                    | What It Cannot Establish Alone          |
|------------------------------|---------------------------------------------------|-----------------------------------------|
| Hand oracle                  | Expected dimensions and selected values           | Behavior on every input                 |
| Public tests                 | Supplied examples and contract checks             | Behavior beyond the supplied cases      |
| Student-written tests        | Independent cases and plausible defects           | Correctness for every possible input    |
| Folder and submission review | Unexpected or missing files in the planned upload | Whether the remaining files are correct |
| Student explanation          | Indices, assumptions, and limitations             | Whether the code actually passed        |

# Submission Rule: Available Records

## Required for Every Assignment

Submit all **reasonably available, user-visible records** of AI coding-tool use related to the assignment. Include unsuccessful or discarded sessions when they can be exported.

- ▶ exportable chats, prompts, follow-ups, and responses;
- ▶ supplied context, attachments, and accepted suggestions;
- ▶ available approvals, errors, retries, and agent action or command logs; and
- ▶ an `AI_USE.md` disclosure of the tool, purpose, and files materially influenced.

**Prompt-to-line and keystroke-level reconstruction are not required.**

If an otherwise expected record is unavailable, state what is missing and why.

# Harness and Agent Records

## Session Identity and Configuration

- ▶ tool/version, displayed model/provider, dates, and mode;
- ▶ supplied instructions and context;
- ▶ relevant configuration, permissions, tools, and integrations; and
- ▶ files or resources made available.

## Action and Verification Evidence

- ▶ available plans, approvals, failures, and retries;
- ▶ available commands, outputs, and tool actions;
- ▶ available tool-generated change summaries or edited-file lists; and
- ▶ verification results and known limitations.

## Sensitive Information

Never submit credentials or protected data. If a required record must be redacted, replace only that value with [REDACTED: category] and leave the rest unchanged.

# Assignment 0 Submission Example

## Submission paths

products.py  
tests/test\_student.py  
report/report.tex  
report/report.pdf  
AI\_USE.md  
ai-records/ when reasonably available

If No AI Coding Tool Was Used

Submit: **I did not use an AI coding tool for this assignment.**

## The evidence connects

- ▶ hand calculations define expected results;
- ▶ student tests target plausible defects;
- ▶ the report records verification commands and results;
- ▶ the planned upload is checked for missing or unexpected files; and
- ▶ AI\_USE.md explains material tool influence or declares no use.

# Three Rules to Remember

1. **Know the roles.** A prompt states the request; a harness controls context, tools, permissions, and records; an agent performs a multi-step workflow.
2. **Treat outputs as candidates.** Inspect the changes and verify them with independent evidence.
3. **Submit available evidence.** Include AI\_USE.md, exportable sessions, submission-file review, and verification results.

# Three Rules to Remember

1. **Know the roles.** A prompt states the request; a harness controls context, tools, permissions, and records; an agent performs a multi-step workflow.
2. **Treat outputs as candidates.** Inspect the changes and verify them with independent evidence.
3. **Submit available evidence.** Include AI\_USE.md, exportable sessions, submission-file review, and verification results.

**You remain responsible for understanding, testing, and explaining all submitted work.**

# Thank You

Any questions?

# References

- C. Autio, R. Schwartz, J. Dunietz, S. Jain, M. Stanley, E. Tabassi, P. Hall, and K. Roberts. Artificial intelligence risk management framework: Generative artificial intelligence profile. Technical Report NIST AI 600-1, National Institute of Standards and Technology, 2024.
- J. Becker, N. Rush, E. Barnes, and D. Rein. Measuring the impact of early-2025 AI on experienced open-source developer productivity. *arXiv preprint arXiv:2507.09089*, 2025.
- P. E. Black, V. Okun, and B. Guttman. Guidelines on minimum standards for developer verification of software. Technical Report NISTIR 8397, National Institute of Standards and Technology, 2021.
- S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer. The impact of AI on developer productivity: Evidence from GitHub Copilot. *arXiv preprint arXiv:2302.06590*, 2023.
- N. Perry, M. Srivastava, D. Kumar, and D. Boneh. Do users write more insecure code with AI assistants? In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pages 2785–2799, 2023. doi: 10.1145/3576915.3623157.
- J. Spracklen, R. Wijewickrama, A. H. M. N. Sakib, A. Maiti, B. Viswanath, and M. Jadliwala. We have a package for you! a comprehensive analysis of package hallucinations by code generating LLMs. In *34th USENIX Security Symposium*, pages 3687–3706. USENIX Association, 2025. URL <https://www.usenix.org/conference/usenixsecurity25/presentation/spracklen>.